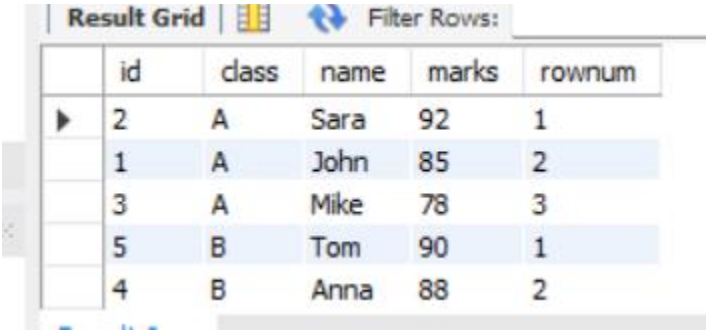
DAY 7 – SQL ASSIGNMENT

```
USE slatask;
drop table students1 ;
-- Table 1
CREATE TABLE students1(
id INT,
class VARCHAR(10),
name VARCHAR(50),
marks INT
);

INSERT INTO students1 VALUES
(1,'A','John',85),
(2,'A','Sara',92),
(3,'A','Mike',78),
(4,'B','Anna',88),
(5,'B','Tom',90);


-- Table 2
CREATE TABLE exam1(
student VARCHAR(50),
subject VARCHAR(20),
score INT
);

INSERT INTO exam1 VALUES
('Alice','Math',90),
('Bob','Math',90),
('Charlie','Math',85),
('David','Science',88),
('Eva','Science',88);
```

```
-- Table 3
CREATE TABLE employees11(
emp_name VARCHAR(50),
department VARCHAR(20),
salary INT
);

INSERT INTO employees11 VALUES
('Alex','IT',7000),
('Brian','IT',7000),
('Chris','IT',6500),
('Diana','HR',6000),
('Eva','HR',5800);

drop table orders ;
-- Table 4
CREATE TABLE orders(
order_id INT,
order_date DATE,
amount INT
);

INSERT INTO orders VALUES
(1,'2024-01-01',100),
(2,'2024-01-02',200),
(3,'2024-01-03',150),
(4,'2024-01-04',300);


-- Table 5
```

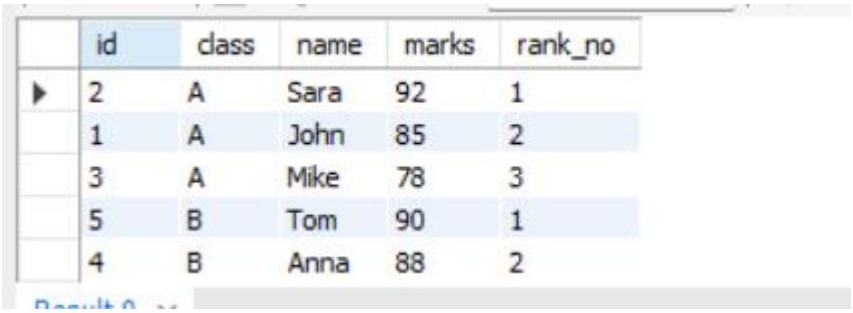
```
CREATE TABLE monthly_sales(
month VARCHAR(10),
sales INT
);

INSERT INTO monthly_sales VALUES
('Jan',5000),
('Feb',6000),
('Mar',5500),
('Apr',7000);
SELECT *,
-- Row Number per Class
SELECT *,
ROW_NUMBER() OVER(PARTITION BY class ORDER BY marks DESC) AS rownum
FROM students1;
```



Result Grid | Filter Rows:

id	class	name	marks	rownum
2	A	Sara	92	1
1	A	John	85	2
3	A	Mike	78	3
5	B	Tom	90	1
4	B	Anna	88	2

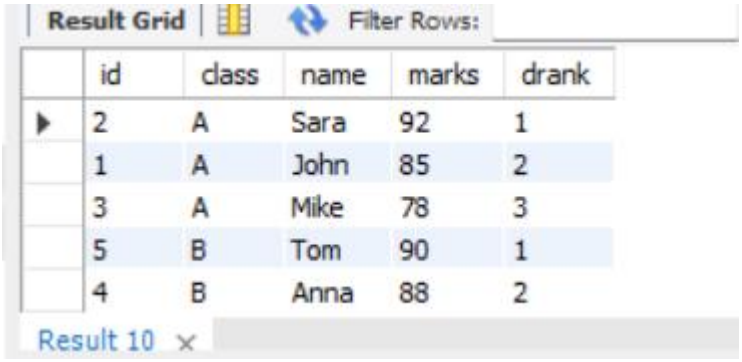
```
-- Rank With Gaps
SELECT *,
RANK() OVER(PARTITION BY class ORDER BY marks DESC) AS rank_no
FROM students1;
```



id	class	name	marks	rank_no
2	A	Sara	92	1
1	A	John	85	2
3	A	Mike	78	3
5	B	Tom	90	1
4	B	Anna	88	2

--3 Rank Without Gaps

```
SELECT id,class,name,marks,
DENSE_RANK() OVER(PARTITION BY class ORDER BY marks DESC) AS drank
FROM students1;
```

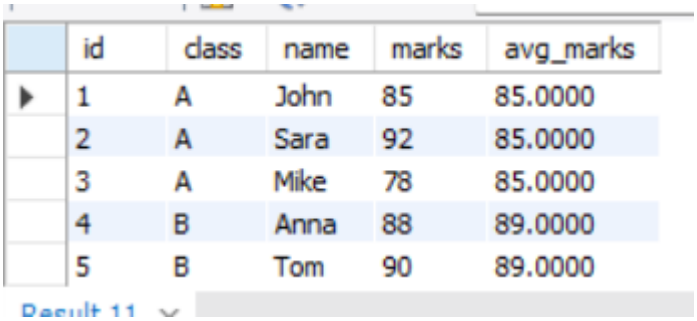


id	class	name	marks	drank
2	A	Sara	92	1
1	A	John	85	2
3	A	Mike	78	3
5	B	Tom	90	1
4	B	Anna	88	2

Result 10

-- 4 Average marks of each class with every student

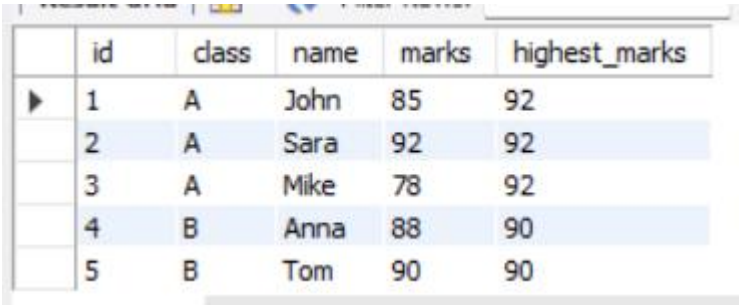
```
SELECT id,class,name,marks,
AVG(marks) OVER(PARTITION BY class) AS avg_marks
FROM students1;
```



id	class	name	marks	avg_marks
1	A	John	85	85.0000
2	A	Sara	92	85.0000
3	A	Mike	78	85.0000
4	B	Anna	88	89.0000
5	B	Tom	90	89.0000

-- 5 Highest marks in each class

```
SELECT id,class,name,marks,
MAX(marks) OVER(PARTITION BY class) AS highest_marks
FROM students1;
```



id	class	name	marks	highest_marks
1	A	John	85	92
2	A	Sara	92	92
3	A	Mike	78	92
4	B	Anna	88	90
5	B	Tom	90	90

-- 6. Row numbers per subject

```
SELECT student,subject,score,
```

```
ROW_NUMBER() OVER(PARTITION BY subject ORDER BY score DESC) AS rownum
FROM exam1;
```

Result Grid | Filter Rows:

student	subject	score	rownum
Alice	Math	90	1
Bob	Math	90	2
Charlie	Math	85	3
David	Science	88	1
Eva	Science	88	2

```
-- 7. Rank per subject (with gaps)
SELECT student,subject,score,
RANK() OVER(PARTITION BY subject ORDER BY score DESC) AS rankno
FROM exam1;
```

Result Grid | Filter Rows:

student	subject	score	rankno
Alice	Math	90	1
Bob	Math	90	1
Charlie	Math	85	3
David	Science	88	1
Eva	Science	88	1

Result 14

```
-- 8. Rank per subject (without gaps)
SELECT student,subject,score,
DENSE_RANK() OVER(PARTITION BY subject ORDER BY score DESC) AS drank
FROM exam1;
```

student	subject	score	drank
Alice	Math	90	1
Bob	Math	90	1
Charlie	Math	85	2
David	Science	88	1
Eva	Science	88	1

```
-- 9. Total students per subject
SELECT student,subject,score,
COUNT(*) OVER(PARTITION BY subject) AS total_students
FROM exam1;
```

student	subject	score	total_students
Alice	Math	90	3
Bob	Math	90	3
Charlie	Math	85	3
David	Science	88	2
Eva	Science	88	2

Result 16

-- 10. Minimum score per subject

```
SELECT student,subject,score,
MIN(score) OVER(PARTITION BY subject) AS min_score
FROM exam1;
```

student	subject	score	min_score
Alice	Math	90	85
Bob	Math	90	85
Charlie	Math	85	85
David	Science	88	88
Eva	Science	88	88

Result 17

-- 11. Row numbers employees per department

```
SELECT emp_name,department,salary,
ROW_NUMBER() OVER(PARTITION BY department ORDER BY salary DESC) AS rownum
FROM employees11;
```

emp_name	department	salary	rownum
Diana	HR	6000	1
Eva	HR	5800	2
Alex	IT	7000	1
Brian	IT	7000	2
Chris	IT	6500	3

-- 12. Rank employees per department

```
SELECT emp_name,department,salary,
RANK() OVER(PARTITION BY department ORDER BY salary DESC) AS rankno
FROM employees11;
```

emp_name	department	salary	rankno
Diana	HR	6000	1
Eva	HR	5800	2
Alex	IT	7000	1
Brian	IT	7000	1
Chris	IT	6500	3

Result 19

-- 13. Rank employees without gaps

```
SELECT emp_name,department,salary,
DENSE_RANK() OVER(PARTITION BY department ORDER BY salary DESC) AS drank
FROM employees11;
```

emp_name	department	salary	drank
Diana	HR	6000	1
Eva	HR	5800	2
Alex	IT	7000	1
Brian	IT	7000	1
Chris	IT	6500	2

-- 14. Total salary per department

```
SELECT emp_name,department,salary,
SUM(salary) OVER(PARTITION BY department) AS total_salary
FROM employees11;
```

emp_name	department	salary	total_salary
Diana	HR	6000	11800
Eva	HR	5800	11800
Alex	IT	7000	20500
Brian	IT	7000	20500
Chris	IT	6500	20500

-- 15. Average salary per department

```
SELECT emp_name,department,salary,
AVG(salary) OVER(PARTITION BY department) AS avg_salary
FROM employees11;
```

emp_name	department	salary	avg_salary
Diana	HR	6000	5900.0000
Eva	HR	5800	5900.0000
Alex	IT	7000	6833.3333
Brian	IT	7000	6833.3333
Chris	IT	6500	6833.3333

-- 16. Row numbers based on order date

```
SELECT order_id,order_date,amount,
ROW_NUMBER() OVER(ORDER BY order_date) AS rownum
FROM orders;
```

order_id	order_date	amount	rownum
1	2024-01-01	100	1
2	2024-01-02	200	2
3	2024-01-03	150	3
4	2024-01-04	300	4

-- 17. Running total of orders

```
SELECT order_id,order_date,amount,
SUM(amount) OVER(ORDER BY order_date) AS running_total
FROM orders;
```

order_id	order_date	amount	running_total
1	2024-01-01	100	100
2	2024-01-02	200	300
3	2024-01-03	150	450
4	2024-01-04	300	750

-- 18. Moving average last 2 orders

```
SELECT order_id,order_date,amount,
AVG(amount) OVER(ORDER BY order_date ROWS 1 PRECEDING) AS moving_avg
FROM orders;
```

order_id	order_date	amount	moving_avg
1	2024-01-01	100	100.0000
2	2024-01-02	200	150.0000
3	2024-01-03	150	175.0000
4	2024-01-04	300	225.0000

-- 19. Maximum order till current row

```
SELECT order_id,order_date,amount,
MAX(amount) OVER(ORDER BY order_date) AS max_amount
FROM orders;
```

order_id	order_date	amount	max_amount
1	2024-01-01	100	100
2	2024-01-02	200	200
3	2024-01-03	150	200
4	2024-01-04	300	300

-- 20. Total number of orders

```
SELECT order_id,order_date,amount,
COUNT(*) OVER() AS total_orders
FROM orders;
```

Result Grid | Filter Rows:

order_id	order_date	amount	total_orders
1	2024-01-01	100	4
2	2024-01-02	200	4
3	2024-01-03	150	4
4	2024-01-04	300	4

-- 21. Row numbers by month

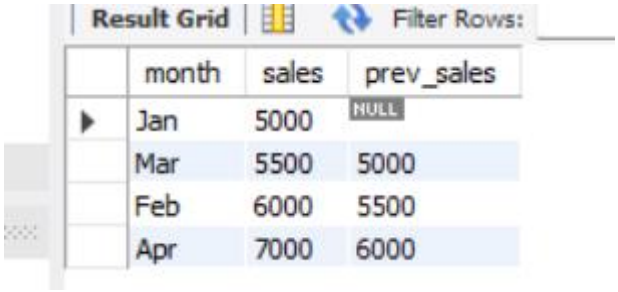
```
SELECT month,sales,
ROW_NUMBER() OVER(ORDER BY sales) AS rownum
FROM monthly_sales;
```

month	sales	rownum
Jan	5000	1
Mar	5500	2
Feb	6000	3
Apr	7000	4

-- 22. Previous month sales

```
SELECT month,sales,
LAG(sales) OVER(ORDER BY sales) AS prev_sales
```

FROM monthly_sales;



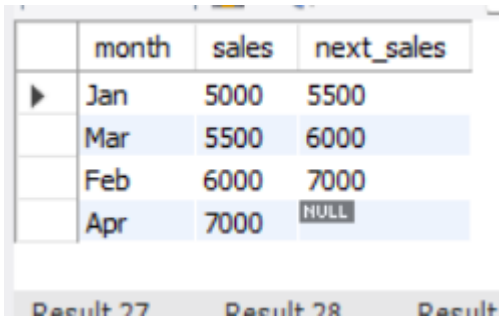
month	sales	prev_sales
Jan	5000	NULL
Mar	5500	5000
Feb	6000	5500
Apr	7000	6000

-- 23. Next month sales

SELECT month,sales,

LEAD(sales) OVER(ORDER BY sales) AS next_sales

FROM monthly_sales;



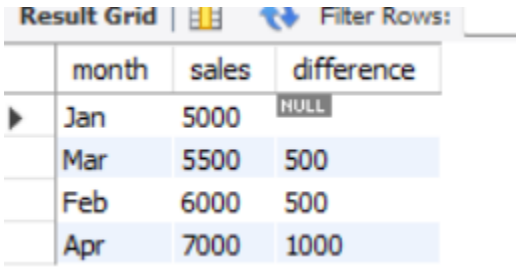
month	sales	next_sales
Jan	5000	5500
Mar	5500	6000
Feb	6000	7000
Apr	7000	NULL

-- 24. Difference from previous month

SELECT month,sales,

sales - LAG(sales) OVER(ORDER BY sales) AS difference

FROM monthly_sales;



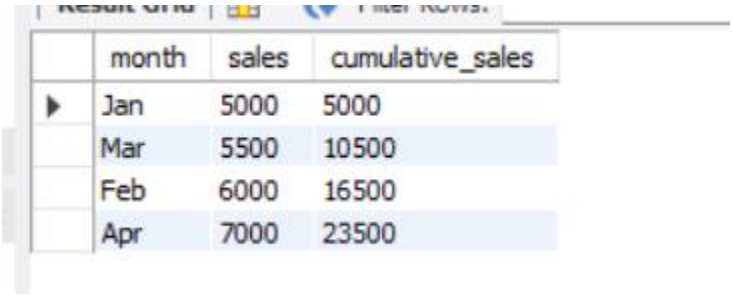
month	sales	difference
Jan	5000	NULL
Mar	5500	500
Feb	6000	500
Apr	7000	1000

-- 25. Cumulative sales

SELECT month,sales,

SUM(sales) OVER(ORDER BY sales) AS cumulative_sales

FROM monthly_sales;



month	sales	cumulative_sales
Jan	5000	5000
Mar	5500	10500
Feb	6000	16500
Apr	7000	23500